

★ TRAINING PROGRAM V1.0

Master **Algorithmic Arbitrage** in 90 Days

A structured, simulator-driven program from data ingestion to SOCPA-compliant trading systems. No external dependencies. No shortcuts. Real professional standards.

"This is not a course. This is a simulation of the most demanding trading desk in the region."

90 Days

3 Phases

~200 Hours

Python 3.8+

Zero Dependencies

CMA / SOCPA / IFRS

⚙ THE CYCLE

How Each Day Works

You receive a briefing, write code against a live mock exchange, then face a ruthless EOD evaluation. Score 100 or you do not advance.



The rule is simple: 100/100 or you repeat the day. There is no partial credit.

★ PREREQUISITES

Before You Begin

You should be comfortable with these Python fundamentals. The AST auditor will catch you if you're not.



Python Syntax

Functions, loops, conditionals, list comprehensions

Day 1



Type Annotations

PEP 484 — `def f(x: int) -> str:`

Day 1

AST Check



Exception Handling

`try / except / finally` blocks

Critical

Day 1



Basic SQL

`SELECT, INSERT, CREATE TABLE`

Day 6



HTTP & JSON

`GET / POST`, status codes, JSON parsing

Day 1



Command Line

Running scripts, navigating directories

Setup

► **If you're not fluent in all of the above:** Start Days 1–3 in *learning mode* (score is informational, not blocking) until you're comfortable enough to attempt a real pass.

① PHASE 1

Data Ingestion & Foundational Risk Math

Days 1–30 — You are a data pipeline. Fetch, calculate, log, repeat. Build reliable data ingestion systems, implement technical indicators from scratch, and master double-entry ledger logging.

DAYS 1–5

REST API Data Ingestion *urllib, JSON, error handling*

SMA

Error Handling

Rate Limiting

Type Annotations

DAY	FOCUS	KEY SKILLS
1	Fetch BTC/USD order book	urllib.request, JSON parsing, try-except
2	Calculate SMA(5) and SMA(20)	Sliding window, list operations, precision
3	Ledger reconciliation	Double-entry verification, P&L calculation
4	Poll with backoff	Exponential backoff, rate limit handling
5	Summary pipeline	30-iteration loop, stdout formatting

DAYS 6–10

Technical Indicators *Indicators from scratch, no libraries*

EMA

Bollinger Bands

RSI

Vectorized Ops

DAY	FOCUS	KEY SKILLS
6	EMA calculation	Exponential weighting, smoothing factor
7	Bollinger Bands	Rolling stdev, upper/lower bands, bandwidth
8	RSI	Gain/loss averaging, overbought/oversold
9	Multi-indicator output	Combined table, cross-validation
10	Indicator suite	All indicators in one pipeline

DAYS 11–20

Ledger Mastery & Risk Metrics

SQLite, double-entry, statistical measures

- SQLite
- Double-Entry
- Sharpe Ratio
- Max Drawdown

DAY	FOCUS	KEY SKILLS
11–15	SQLite schema & aggregation	Schema design, JOINS, GROUP BY, window functions
16–20	Statistical risk metrics	Variance, stdev, Sharpe ratio, max drawdown

DAYS 21–30

Pipeline Architecture & Capstone *Production-ready data pipelines*

- Batching
- Windowing
- Streaming
- Capstone

DAY	FOCUS	KEY SKILLS
21–25	Pipeline optimization	Memory-efficient streaming, batching strategies
26–30	Phase 1 Capstone	End-to-end ingestion → calculation → logging. All critical AST checks must pass.

Common failure modes in Phase 1:

- Missing try-except on network calls — automatic 0
- O(n²) nested loops on large datasets — AST warning → -10 pts
- Ledger imbalance (debits ≠ credits) — test failure → -30 pts

② PHASE 2

Arbitrage Logic

Days 31–60 — Find the spread. Execute before it's gone. Cross-exchange detection, triangular arbitrage cycles, position sizing, and real-time execution.

DAYS 31–40

Cross-Exchange Arbitrage *Spread detection, fee-aware execution*

Spread Detection

Execution

Position Sizing

Fee Modeling

DAY	FOCUS	KEY SKILLS
31–35	Cross-exchange detection	Poll PRIMARY vs SECONDARY, calculate spread, threshold detection
36–40	Arbitrage execution	POST /v1/execute, fee-aware P&L, fill tracking

DAYS 41–55

Triangular Arbitrage & Risk Controls

Multi-asset cycles, Kelly Criterion, slippage

Triangular Arb

Kelly Criterion

Stop-Loss

Slippage

DAY	FOCUS	KEY SKILLS
41–45	Triangular arbitrage	3-asset cycles (BTC→ETH→USDT→BTC), rate conversion, opportunity scanning
46–50	Risk controls	Kelly Criterion sizing, stop-loss, max exposure
51–55	Latency & slippage	Timestamp diff analysis, fill probability, slippage models

DAYS 56–60

Phase 2 Capstone *Real-time triangular scanner with auto-execution*

Capstone

Real-Time

Auto-Execute

Reconciliation

DAY	FOCUS	KEY SKILLS
56– 60	Phase 2 Capstone	Real-time triangular scanner with auto-execution, risk limits, full ledger reconciliation

Common failure modes in Phase 2:

- Trading without checking spread threshold — logic error → test failure
- Executing without sufficient balance — ledger inconsistency
- Missing fee calculations in P&L — SOCPA compliance violation
- No rate limiting on exchange calls — simulated ban → day reset

③ PHASE 3

Slippage & SOCPA Compliance

Days 61–90 — It's not enough to be profitable. You must be provably correct. Audit trails, IFRS 9 reporting, VaR & CVaR, and a final 5-day audit simulation.

DAYS 61–70

SOCPA Audit Trails & IFRS 9

Compliance and financial instrument reporting

- SOCPA
- IFRS 9
- Audit Trails
- Disclosures

DAY	FOCUS	KEY SKILLS
61–65	SOCPA audit trails	Provenance records, timestamps, authorization chains
66–70	IFRS 9 reporting	Instrument classification, impairment, disclosure notes

DAYS 71–85

VaR, CVaR & System Resilience

Risk analytics and hardened systems

- VaR
- CVaR
- Stress Testing
- Resilience

DAY	FOCUS	KEY SKILLS
71–75	VaR & CVaR	Historical simulation VaR (95%/99%), expected shortfall, stress scenarios
76–80	Compliance reports	Automated report generation, auditor-ready output
81–85	System resilience	Graceful degradation, outage handling, data recovery

DAYS 86–90

Final Capstone

5-day external audit simulation

- Capstone
- Audit
- Compliance
- Reconciliation

DAY	FOCUS	KEY SKILLS
86–90	Final Capstone	Full audit simulation: run all 90 days, produce complete compliance package, pass external audit

Failure in Phase 3 means:

- Missing audit timestamps — **automatic 0, SOCPA violation**
- IFRS classification errors — test failure → -30 pts
- VaR calculation off by >1% — precision failure → day reset
- Any ledger imbalance after reconciliation — compliance breach
- System crash during exchange downtime — resilience failure

 SCHEDULE

Weekly Rhythm

Each week follows a predictable cadence. Total commitment: 4–8 hours per week for most participants.

MONDAY Receive Briefing 15 min	TUESDAY Write Solution 1–3 hrs	WEDNESDAY Debug & Iterate 1–2 hrs	THURSDAY EOD & Fix 1–2 hrs
FRIDAY Advance & Review 30 min	SATURDAY Optional: Refactor —	SUNDAY Rest / Catch Up —	

SCORING

Score = max(0, 100 – deductions)

You start at 100. Every violation or failure deducts from your score. Two critical AST violations = automatic 0. You need 100 to advance.

COMPONENT	DEDUCTION	DESCRIPTION
• AST Critical Violation	-40	No try-except on network calls, eval/exec, prohibited imports. Two = automatic 0.
• AST Warning	-10	Nested loops, missing annotations, division by zero, infinite loops, global state
• Test Failure	-30	Hidden test suite checks functional correctness of your solution
• Ledger Imbalance	-30	Double-entry verification failure (debits ≠ credits)
• Manual Deductions	Variable	Principal Strategist may apply situational penalties for egregious violations

1 CRITICAL

60

Score = 100 – 40

2 CRITICALS

0

Automatic zero, day reset

1 WARNING

90

Not enough to advance

PERFECT DAY

100

You advance

⚙️ AST AUDITOR

What Gets Checked

The AST auditor parses your Python code into an abstract syntax tree and runs 9 pattern-matching checks. Every EOD evaluation runs through this gauntlet.

● CRITICAL

-40

NO_NETWORK_EXCEPT

`urllib.request.urlopen()` or `Request()` without `try`.
Exchange goes down = your pipeline crashes.

● CRITICAL

-40

EVAL_EXEC

`eval(...)` or `exec(...)` detected. Arbitrary code execution is a security risk and SOCPA violation.

● CRITICAL

-40

PROHIBITED_IMPORT

`pickle`, `shelve`, `marshal` — deserialization vulnerabilities that compromise audit integrity.

● WARNING

-10

NESTED_LOOP

`for i in range(n): for j in range(m):` — $O(n^2)$ complexity that doesn't scale to production datasets.

● WARNING

-10

DIVISION_BY_ZERO

`a / b` without `a != 0` guard. Runtime crash waiting to happen.

● WARNING

-10

INFINITE_WHILE

`while True:` without a `break` statement. Never-terminating loop that hangs the pipeline.

● WARNING

-10

GLOBAL_STATE

`global x` at module level. Side effects make code untestable and unpredictable.

● INFO

-0

MISSING_ANNOTATION

Function parameter or return value without a type hint. PEP 484 compliance is expected.

 STANDARDS

Three Professional Frameworks

The simulator enforces real-world accounting and financial standards. Understanding them is part of the training.

CMA

Certified Management Accountant

Part 1: Financial Planning, Performance & Analytics

Section C: Financial Statement Analysis

Section E: Investment Decisions

Part 2: Strategic Financial Management

Section A: Financial Statement Analysis

Section D: Risk Management

SOCPA

Saudi Org. for Chartered & Professional Accountants

Standard 4: Documentation and audit trail requirements

Standard 7: Financial instrument recognition and measurement

Standard 12: Revenue recognition and trade recording

IFRS

International Financial Reporting Standards

IFRS 9: Financial Instruments — classification, measurement, impairment

IFRS 13: Fair Value Measurement — hierarchy levels, disclosure

IFRS 15: Revenue from Contracts with Customers — trade revenue recognition

 RESOURCES

Everything You Need

The simulator uses only Python's standard library. External concepts link to free references.

Python Standard Library

[urllib.request](#)

HTTP GET/POST to mock exchange

Days 1–60

[urllib.error](#)

HTTP error handling

Days 1–60

[json](#)

Parse exchange responses

Days 1–90

[sqlite3](#)

Ledger database

Days 1–90

[math](#)

Statistical calculations

Days 11–30, 71–75

[statistics](#)

Mean, stdev for risk metrics

Days 16–20

[ast / threading / subprocess](#)

Used internally by simulator (understanding only)

—

External Concepts

[SMA / EMA](#)

Days 1–10 — Investopedia: Moving Average

[Bollinger Bands](#)

Days 6–10 — Investopedia: Bollinger Bands

[Sharpe Ratio](#)

Days 16–20 — Wikipedia: Sharpe Ratio

[Cross-Exchange Arbitrage](#)

Days 31–35 — QuantConnect: Arbitrage

[Triangular Arbitrage](#)

Days 41–45 — Binance Academy

[Kelly Criterion](#)

Days 46–50 — Investopedia

Value at Risk (VaR)

Days 71-75 — CFA Institute

CVaR / Expected Shortfall

Days 71-75 — Wikipedia

 PROGRESS

Tracking Your Journey

The built-in status command shows your cumulative record. The goal is a perfect run.

DAY 30 — END OF PHASE 1

Data Ingestion & Risk Math

30/30 days passed · 3000/3000 cumulative score · All critical AST checks passing

DAY 60 — END OF PHASE 2

Arbitrage Logic

60/60 days passed · 6000/6000 cumulative score · Real-time scanner operational

DAY 90 — GRADUATION

SOCPA Compliance & Final Capstone

90/90 days passed · 9000/9000 cumulative score · Compliance package generated

 GRADUATION

What It Takes to Finish

The program is designed to be brutal. Finishing is a genuine achievement.



All 90 Days at 100

Every single day passed with a perfect score. No warnings, no criticals.



Zero Critical Violations

Every network call wrapped, no eval/exec, no prohibited imports across all 90 days.



Ledger Always Balanced

Double-entry bookkeeping verified after every single day. Debits always equal credits.



Compliance Package Generated

Final capstone produces a complete, auditor-ready compliance report covering all 90 days.

Graduates receive:

Completion certificate (generated by the simulator on Day 90)

Their name added to the NovaCap "Honor Roll"

The satisfaction of surviving the most demanding Python training simulation ever built

*"This is not a course. This is a simulation of
the most demanding trading desk in the region."*

— Principal Strategist, NovaCap Financial Technologies Ltd.

Principal Strategist — 90-Day FinTech Arbitrage Training Program v1.0.0

[Product Details](#) · [License Agreement](#) · [Landing Page](#) · [Live Dashboard](#)

Python Standard Library · CMA © IMA · SOCPA © SOCPA · IFRS © IFRS Foundation